



De l'énoncé au code

4PYTH – Correction guidée du défi 2

Chris CHEVALIER



La méthode en 4 étapes

1. Comprendre

Qu'est-ce qui entre dans chaque fonction ?
Qu'est-ce qui en sort ? Que donnent les exemples de l'énoncé ?

2. Découper

Écrire les étapes **en français**, une par ligne, avant de taper la moindre ligne de code. C'est le **pseudo-code**.

3. Coder petit à petit

Une fonction à la fois. Le programme doit tourner après chaque ajout, même s'il ne fait pas encore tout.

4. Tester tout de suite

Relancer avec les exemples de l'énoncé après chaque fonction. Une erreur trouvée tôt est une erreur facile à corriger.



Étape 1 – Comprendre l'énoncé

Fonction	Entrée	Sortie
<code>count_failures(lines)</code>	la liste des lignes du journal	un dictionnaire {IP: nombre d'échecs}
<code>detect_brute_force(lines, threshold=5)</code>	les lignes et un seuil	une liste d'IP
<code>print_report(lines, threshold=5)</code>	les lignes et un seuil	rien : elle affiche une alerte par IP

Les trois fonctions s'appuient les unes sur les autres. Pour détecter, il faut d'abord **compter**.
Pour afficher le rapport, il faut d'abord **détecter**.



Étape 1 – Lire une ligne du journal

```
Deux lignes Failed password
Oct  7 10:02:11 srv-web sshd[2210]: Failed password for root from 203.0.113.45 port 51122 ssh2
Oct  7 10:02:18 srv-web sshd[2215]: Failed password for invalid user admin from 203.0.113.45 port 51141 ssh2
```

```
Terminal
>>> line = AUTH_LOG[3]
>>> words = line.split()
>>> words.index("from")
9
>>> words[words.index("from") + 1]
'203.0.113.45'
```

- `split()` découpe la ligne en **mots**, quel que soit le nombre d'espaces
- La 2e ligne a deux mots de plus (« invalid user »). L'IP n'est donc **pas toujours à la même position**
- Elle suit **toujours** le mot `from`. On cherche donc la position de `from` puis on prend le mot suivant



Étape 2 – Le plan en français

Pseudo-code

```
extraire l'IP d'une ligne :  
    découper la ligne en mots, renvoyer le mot qui suit « from »  
  
compter les échecs :  
    créer un dictionnaire vide  
    pour chaque ligne qui contient « Failed password » :  
        extraire l'IP puis ajouter 1 à son compteur  
    renvoyer le dictionnaire  
  
détecter le brute force :  
    pour chaque IP et son nombre d'échecs : si le nombre atteint le seuil, garder l'IP  
  
afficher le rapport :  
    pour chaque IP détectée : afficher une alerte avec son nombre d'échecs
```



Étape 3a – Extraire l'IP

```
challenge2.py

from log_data import AUTH_LOG

def extract_ip(line):
    """Renvoie l'IP qui suit le mot « from »."""
    words = line.split()
    return words[words.index("from") + 1]

print(extract_ip(AUTH_LOG[3])) # 203.0.113.45
print(extract_ip(AUTH_LOG[5])) # 203.0.113.45
```

- Une fonction **courte**, testée tout de suite sur une ligne simple et sur une ligne « invalid user »
- Le journal est rangé dans `log_data.py` et importé. Le fichier du défi reste lisible
- On pourra la réutiliser partout où il faut une IP, y compris pour les bonus



Étape 3b – Compter les échecs

```
challenge2.py (suite)

def count_failures(lines):
    """Compte les échecs par adresse IP."""
    failures = {}
    for line in lines:
        if "Failed password" in line:
            ip = extract_ip(line)
            failures[ip] = failures.get(ip, 0) + 1
    return failures
```

- "Failed password" in line garde seulement les échecs. Les lignes Accepted ou Connection closed sont ignorées
- failures.get(ip, 0) renvoie le compteur actuel ou 0 si l'IP est nouvelle
- Le return est **après** la boucle, une fois toutes les lignes lues



Étape 3c – Détecter le brute force

```
challenge2.py (suite)

def detect_brute_force(lines, threshold=5):
    """Renvoie les IP ayant au moins threshold échecs."""
    suspects = []
    for ip, count in count_failures(lines).items():
        if count >= threshold:
            suspects.append(ip)
    return suspects
```

- On **réutilise** `count_failures()` au lieu de recompter
- `items()` donne chaque paire clé et valeur : l'IP et son nombre d'échecs
- `>=` et non `>` : l'énoncé dit « **au moins** » `threshold` échecs



Étape 3d – Afficher le rapport

```
challenge2.py (suite)

def print_report(lines, threshold=5):
    """Affiche une alerte par IP détectée."""
    failures = count_failures(lines)
    for ip in detect_brute_force(lines, threshold):
        message = f"{failures[ip]} échecs d'authentification"
        print(f"[ALERTE] {ip} : {message}")
```

- On a besoin **des deux** : la liste des IP détectées et leur nombre d'échecs
- Le seuil reçu est **transmis** à `detect_brute_force()`. Sinon, `print_report(lines, 2)` utiliserait toujours 5
- Cette fonction **affiche** et ne renvoie rien



Étape 4 – Tester avec les exemples

```
Tests

print(count_failures(AUTH_LOG))
# {'192.0.2.25': 1, '203.0.113.45': 7, '198.51.100.23': 5, '198.51.100.77': 2}
print(detect_brute_force(AUTH_LOG))
# ['203.0.113.45', '198.51.100.23']
print(detect_brute_force(AUTH_LOG, threshold=2))
# ['203.0.113.45', '198.51.100.23', '198.51.100.77']
print_report(AUTH_LOG)
# [ALERTE] 203.0.113.45 : 7 échecs d'authentification
# [ALERTE] 198.51.100.23 : 5 échecs d'authentification
```

Si un résultat diffère, on relance la fonction concernée seule. Comme chaque fonction est courte, l'erreur se trouve vite.



Les pièges les plus fréquents

Piège	Correction
l'IP prise à une position fixe (<code>words[9]</code>)	chercher la position de <code>from</code> avec <code>index()</code>
<code>failures[ip] += 1</code> sur une IP nouvelle	<code>KeyError</code> . Utiliser <code>get(ip, 0)</code>
<code>count > threshold</code>	l'énoncé dit « au moins ». Utiliser <code>>=</code>
tout le code dans une seule fonction	une fonction par tâche, qui réutilise les autres
le seuil oublié dans <code>print_report()</code>	le transmettre à <code>detect_brute_force()</code>

Découper un problème en **petites fonctions** le rend plus simple à écrire, à tester et à réutiliser.



Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com